

MEDIATEK

MT8676 Reserved Memory User Manual

Version: 1.1
Release date: 2024-11-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-10-28	Lixin Li	Official release
1.1	2024-11-12	Lixin Li	Added the description of suitable frame into Section 1.1 Overview

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

 1.1 Overview 4

 1.2 How to Add a New Reserved Memory? 4

 1.2.1 How to Add a New Reserved Memory Node in Kernel dts? 4

 1.2.2 How to Use mblock to Reserve Memory? 5

 1.3 How Does the Kernel Driver Obtain Reserved Memory for Use? 7

 1.4 How to Free a Reserved Memory? 8

Exhibit 1 Terms and Conditions 9

List of Figures

Figure 1-1. Diagram of the relationship between reserved memory and Memtotal 4

Figure 1-2. mblock_alloc usage example 6

Figure 1-3. mblock_query_reserved_by_name usage example 7

Figure 1-4. Kernel driver gets reserved memory example 7

Figure 1-5. mblock_free usage example 8

Figure 1-6. free_reserved_page usage example 8

1 Reserved Memory

1.1 Overview

For Yocto and Android, the meaning and usage of reserved memory are basically the same. This text is suitable for LA/LLA frame.

In short, it refers to a portion of memory resources managed by the Kernel that is reserved for the Kernel system itself or for other module functions. The reserved memory can be used independently by specific modules without interference from other modules. However, this also results in a reduction of the overall dynamically allocatable memory for the system.

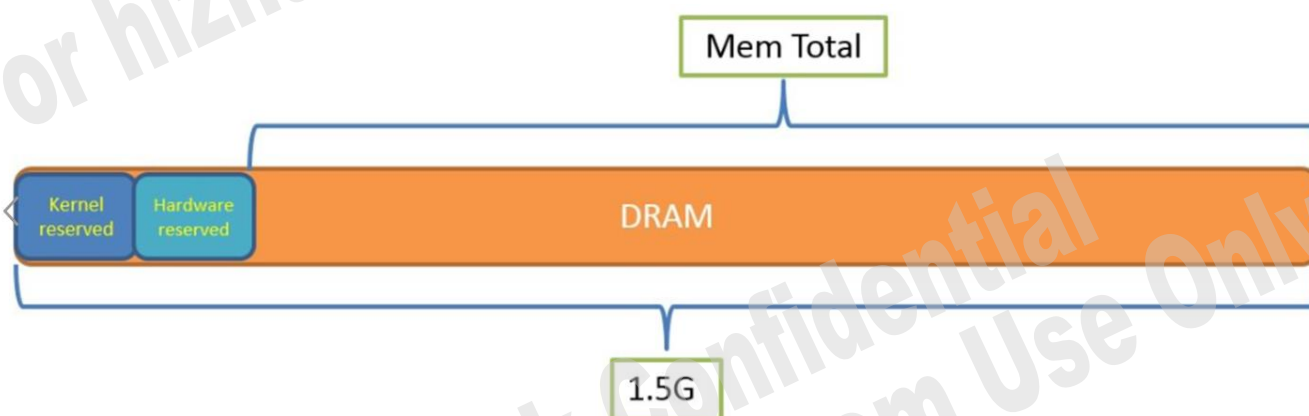


Figure 1-1. Diagram of the relationship between reserved memory and Memtotal

HW Dram size = MemTotal + HW Reserved Memory + Kernel Reserved Memory

Reserved memory mainly includes HW module reserved memory and Kernel reserved memory:

1. HW module reserved memory: This is a portion of memory reserved for certain hardware or features on the platform (such as TEE, SCP, display, etc.) for their independent use.
2. Kernel reserved memory: This is memory reserved by the Kernel for storing its code, data, and critical functional data structures (such as struct page) and buffers.

1.2 How to Add a New Reserved Memory?

There are currently two ways to add a new reserved memory:

1. Add a new reserved memory node in the Kernel dts file;
2. Call the `mblock` API in LK2 to add a new reserved memory (recommended).

1.2.1 How to Add a New Reserved Memory Node in Kernel dts?

Find the reserved memory node in Kernel dts and add your own reserved memory child node under it.

```
reserved-memory {
    xxx-reserved-memory {
```

```

compatible = "xxx";
no-map;
size = <0 0x200000>;
alignment = <0 0x200000>;
};
xxx-reserved-memory {
    compatible = "xxxx";
    reg = <0 0x44400000 0 0x10000>;
};
.....
};

```

- **The blue words:** The start address of the reserved memory;
- **The red words:** The size;
- **alignment:** The alignment size;
- **compatible:** Associate the corresponding driver, and execute the corresponding driver to perform special processing on this reserved memory during the Kernel startup process;
- **no-map:** Adding this attribute means that this memory will not be linearly mapped from PA → VA, and the Kernel will not manage this memory;

This is a method natively provided by the Kernel. For more information on the meanings of various nodes, please refer to https://android.googlesource.com/kernel/msm/+android-7.1.0_r0.2/Documentation/devicetree/bindings/reserved-memory/reserved-memory.txt

1.2.2 How to Use mblock to Reserve Memory?

mblock is a mechanism developed by MediaTek to manage reserved memory in LK2. The APIs used for adding a reserved memory are `mblock_alloc` and `mblock_alloc_range`.

The following parameters need to be specified when using `mblock`:

- **reserved_size:** Reserved memory size;
- **align:** Reserved memory alignment size;
- **lower_bound:** Reserved memory address lower limit;
- **limit:** Reserved memory address upper limit;
- **expected_address:** Specify the expected start address. If it is free, it will be assigned to this start address; if it is already occupied, an error will be reported. If `expected_address` is not specified, a range that meets the size requirements will be dynamically found based on the upper and lower limits of the address.
- **mapping:** Reserved memory mapping type, 0 is no-map, 1 is mapping, 2 is reusable;
- **name:** The name of the reserved memory;
- **mblock_alloc direction:** By default, it will search from `limit` to `lower_bound~limit`, and return a free address that meets the requirements; if no address that meets the requirements is found, it will fail.

The example is as follows:

```

/* allocate apusys-rv_secure */
if (need_relocate())
    sec_mem_addr_pa = mblock_alloc_range(sec_mem_size, APUSYS_SEC_MEM_ALIGN,
        APUSYS_MEM_LOWER_BOUND, APUSYS_MEM_UPPER_BOUND, 0, 0, "apu_apusys-rv_secure");
else
    sec_mem_addr_pa = mblock_alloc(sec_mem_size, APUSYS_SEC_MEM_ALIGN, APUSYS_MEM_LIMIT,
        0, 0, "apu_apusys-rv_secure");

if (!sec_mem_addr_pa) {
    dprintf(CRITICAL, "apu_apusys-rv_secure mblock allocation failed\n");
    ret = ERR_NO_MEMORY;
    goto err_apusys_rv_sec_alloc_fail;
}

/* set vmm */
ret = vmm_alloc_physical(vmm_get_kernel_aspace(),
    "vm-apu_apusys-rv_secure",
    sec_mem_size,
    &sec_mem_addr_va,
    PAGE_SIZE_SHIFT,
    (paddr_t)sec_mem_addr_pa,
    0,
    ARCH_MMU_FLAG_CACHED);

if (ret != NO_ERROR) {
    dprintf(CRITICAL, "APUSYS vmm_alloc_physical vm-apu_apusys-rv_secure failed, err: %d\n",
        ret);
    goto err_vmm_apu_apusys_rv_secure_alloc;
}

memset(sec_mem_addr_va, 0, sec_mem_size);

apusys_rv_initialize_sec_buf(sec_info_tmp, part_tmp_buf, sec_mem_addr_va, sec_mem_size);

```

Figure 1-2. mblock_alloc usage example

If alloc succeeds, the following log will be displayed. Start: 0x8fa00000 is the start address of alloc.

```
mblock_alloc_range_no_lock:535: start: 0x8fa00000, sz: 0x600000 lower_bound: 0x0, limit:
0x90000000, mblock_alloc_range_no_lock:537: map:0 name:apu_apusys-rv_secure
```

1. The principle of allocating reserved memory using `mblock_alloc` is the same as that of reserved memory in dts. At the end of LK2, the information of reserved memory allocated by `mblock_alloc` will be written into the fdt as dts nodes and the corresponding log is as follows:

```
mblock_fdt_reserved_append:1595: mblock-reserved-memory is appended successfully
```

2. In other files of LK2, if you need to query previously reserved memory, you can use the `mblock_query_reserved_by_name(xxx, 0)` function, where the parameter `xxx` is the name of the reserved memory. This function returns a `Reserved` structure, in which `reserved->size` indicates the size of the queried memory, and `reserved->start` indicates the start address of the queried memory.

The example is as follows:

```
int aee_debug_kinfo_map(void)
{
    const struct reserved_t *reserved = NULL;
    status_t err;

    reserved = mblock_query_reserved_by_name("aee_debug_kinfo", 0);
    if (reserved) {
        err = vmm_alloc_physical(vmm_get_kernel_aspace(),
                                "aee_debug_kinfo", reserved->size,
                                &aee_debug_kinfo_va, 0,
                                reserved->start, 0, ARCH_MMU_FLAG_CACHED);
        if (err) {
            voprintf_error("vmm_alloc_physical for memmap fail %d\n", err);
            return -1;
        }
    } else {
        voprintf_error("query aee_debug_kinfo fail\n");
        return -1;
    }

    return 0;
}
```

Figure 1-3. mblock_query_reserved_by_name usage example

1.3 How Does the Kernel Driver Obtain Reserved Memory for Use?

In the Kernel driver, find the specified node through the compatible name, parse the start physical address and size of the corresponding reserved memory, and then map it to a virtual address. After that, you can use this memory.

The example is as follows:

```
if ((apu->platdata->flags & F_SECURE_BOOT) == 0) {
    apusys_sec_mem_node = of_find_compatible_node(NULL, NULL,
        "mediatek,apu_apusys-rv_secure");
    if (!apusys_sec_mem_node) {
        apu_drv_debug("DT,mediatek,apu_apusys-rv_secure not found\n");
        ret = -EINVAL;
        goto out_free_rproc;
    }
    of_property_read_u64_index(apusys_sec_mem_node, "reg", 0,
        &(apu->apusys_sec_mem_start));
    of_property_read_u64_index(apusys_sec_mem_node, "reg", 1,
        &(apu->apusys_sec_mem_size));
    apu_drv_debug("%s: start = 0x%llx, size = 0x%llx\n",
        apusys_sec_mem_node->full_name, apu->apusys_sec_mem_start,
        apu->apusys_sec_mem_size);
    apu->apu_sec_mem_base = memremap(apu->apusys_sec_mem_start,
        apu->apusys_sec_mem_size, MEMREMAP_WC);
}
```

Figure 1-4. Kernel driver gets reserved memory example

1.4 How to Free a Reserved Memory?

In the LK2 stage, the APIs used to free a block of reserved memory are `mblock_free` and `mblock_free_partial`.

`int mblock_free(u64 addr):` Free the entire reserved memory starting with `addr`.

`int mblock_free_partial(u64 addr, u64 size):` Free the reserved memory starting at `addr` with a size of `size`.

The following parameters should be specified when using the APIs:

- `addr`: If the whole block is freed, fill in the `start addr` of the reserved memory returned by `alloc`; if part of the block is freed, fill in the `start addr` of the reserved memory that you want to free;
- `size`: The size to be freed.

The usage example is as follows:

```
void plat_memory_reserve_free(void)
{
    mblock_free(LK_KERNEL_64_BASE_PHY);
    mblock_free(LK_DT_BASE_PHY);
    mblock_free(LK_RAMDISK_BASE_PHY);
}
```

Figure 1-5. `mblock_free` usage example

The corresponding log on successful free is as follows:

```
mblock_free_with_size:970: start 0x1c0000000 size: 0x7c00000, name: LK_KERNEL
```

Note:

- Currently, the `mblock`-related functions `mblock_alloc` and `mblock_free` are only supported for use in the LK2 stage. If the reserved memory is used up in the LK stage, it can be freed using `mblock_free`.
- If it needs to be retained for use in the Kernel stage for a period of time before being freed, the API `free_reserved_page` can be called after the Kernel has started to free it.

An example is as follows.

```
static int free_reserved_buf(phys_addr_t start_phys, phys_addr_t end_phys)
{
    phys_addr_t pos;

    WARN_ON(start_phys & ~PAGE_MASK);
    WARN_ON(end_phys & ~PAGE_MASK);

    if (end_phys <= start_phys) {
        DDPPR_ERR("%s end_phys:0x%x is smaller than start_phys:0x%x\n",
            __func__, (unsigned long)end_phys, (unsigned long)start_phys);
        return -1;
    }

    for (pos = start_phys; pos < end_phys; pos += PAGE_SIZE)
        free_reserved_page(phys_to_page(pos));

    return 0;
}
```

Figure 1-6. `free_reserved_page` usage example

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.